

Model-Free Prediction and Control

Learning policies without the environment model — Lecture 3/4

Etienne Chassaing

CentraleSupélec

1. Introduction
2. Monte Carlo Learning
3. TD Learning
4. On-Policy Control
5. Off-Policy Control
6. Summary

1. Introduction
2. Monte Carlo Learning
3. TD Learning
4. On-Policy Control
5. Off-Policy Control
6. Summary

By the end of this lecture, you should understand:

1. **Monte Carlo methods** — learn from complete trajectories
2. **Temporal Difference (TD) learning** — bootstrap and learn from partial trajectories
3. **Control algorithms** — SARSA (on-policy), Q-learning (off-policy)
4. **Bias-Variance trade-off** — MC vs TD vs Dynamic Programming
5. **Practical applications** — when to use each algorithm

Context: Lectures 1–2 assumed we knew the model. Here, we learn both prediction (estimate V) and control (find optimal π) by interacting only.

Lecture 1–2: Planning

- Knew the model $p(s'|s, a)$
- Solved Bellman equations exactly
- Computed V_* or π_* in closed form

Two learning phases:

- **Prediction:** Given a policy π , learn its value function V^π
- **Control:** Learn to act optimally — find π_* through exploration

Lecture 3: Model-Free Learning

- Model unknown
- Learn from interaction (trial-and-error)
- Estimate V^π (prediction) or Q_* (control)
- Discover optimal policy by exploring

Goal: Given a fixed policy π , estimate its value function $V^\pi(s)$ — *without* knowing $p(s'|s, a)$.

Why we can't just apply Lecture 2's methods:

- Policy evaluation (Lecture 2) needs $p(s'|s, a)$ to compute the Bellman expectation exactly
- Here the model is unknown — we only get to *sample* trajectories by interacting

Idea: Replace the expectation over the model with an average over *experience*.

- Monte Carlo: average full-episode returns G_t
- TD: bootstrap from one-step samples $r_{t+1} + \gamma V(s_{t+1})$

→ Two families of prediction algorithms, covered next.

Goal: Find (near-)optimal policy π_* — again without knowing $p(s'|s, a)$.

Why V is not enough anymore:

- Policy improvement needs $\arg \max_a \sum_{s', r} p(s', r|s, a)[r + \gamma V(s')]$ — requires the model
- Solution: learn $Q(s, a)$ instead of $V(s)$, so improvement becomes $\arg \max_a Q(s, a)$ — model-free

New ingredient: exploration.

- We must try actions we don't yet believe are best, or we can get stuck on a suboptimal policy
- Typically handled with ϵ -greedy policies

Two families of control algorithms:

- **On-policy** (SARSA): learn about the policy you follow
- **Off-policy** (Q-learning): learn about the optimal policy while following another

1. Introduction
- 2. Monte Carlo Learning**
3. TD Learning
4. On-Policy Control
5. Off-Policy Control
6. Summary

Core idea (not specific to RL): Approximate a quantity you cannot compute exactly by an **average over repeated random experiments**.

General recipe:

1. Want to know $\mathbb{E}[X]$ for some random variable X , but no closed form
2. Sample many independent realisations $x_1, x_2, \dots, x_N$ of X
3. Estimate: $\mathbb{E}[X] \approx \frac{1}{N} \sum_{i=1}^N x_i$

Why it works: Law of large numbers — the sample average converges to the true expectation as $N \rightarrow \infty$.

Classic examples: estimating π by random sampling, pricing options by simulating price paths, integrating a function by random sampling.

In RL: X = the return G_t obtained by playing out an episode; we don't know its distribution, so we sample episodes and average.

Monte Carlo Prediction

Idea: Instance of the general MC recipe — $X = G_t$, the return. Average returns from complete episodes where we visit state s .

Algorithm (first-visit MC):

1. Generate an episode following policy π : $s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T$
2. For each state s visited (first time it appears in the episode), compute the return that followed:

$$G_t = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{T-t-1} r_T$$

3. Append G_t to a list $\text{Returns}(s)$; set $V(s) \leftarrow \text{average}(\text{Returns}(s))$

First-visit vs every-visit: a state s can be visited several times within one episode. The *first visit* to s is the first time it occurs in that episode.

- **First-visit MC** estimates $V^\pi(s)$ as the average of the returns that follow *first* visits to s
- **Every-visit MC** averages the returns that follow *all* visits to s

Incremental form (no need to store all returns):

$$V(s) \leftarrow V(s) + \frac{1}{N(s)} [G_t - V(s)], \quad N(s) \leftarrow N(s) + 1$$

With constant step size α :

$$V(s) \leftarrow V(s) + \alpha [G_t - V(s)]$$

tracks a moving average, useful when π or the environment drifts (older returns are gradually forgotten).

Properties:

- Unbiased (first-visit): G_t is a true sample of $V^\pi(s)$
- High variance: depends on the randomness of the whole episode
- Needs full episodes before any update — no online learning

Example: Blackjack (Sutton & Barto)

Setup: a player draws cards trying to get closer to 21 than the dealer without going over. **State** = (player's current sum 12–21, dealer's showing card ace–10, does the player hold a usable ace) $\rightarrow$ 200 states. **Actions:** `hit` (draw) or `stick` (stop). **Reward:** +1 win, -1 lose, 0 draw — given only at the terminal step; $\gamma = 1$, episodic.

Fixed policy evaluated: stick if player's sum ≥ 20 , otherwise hit.

Why Monte Carlo, not DP? The dealer's card-drawing dynamics are known in principle, but encoding $p(s'|s, a)$ exactly (accounting for every possible card sequence) is tedious and error-prone. It is far easier to *simulate* games and let MC average the outcomes — no model needed.

Procedure: play many games under π ; for each state visited, apply first-visit MC to accumulate returns and estimate $V^\pi(s)$ for all 200 states.

Example: Blackjack — Results

Sutton & Barto estimate V^π after 10,000 and 500,000 episodes, separately for states with and without a usable ace.

- **After 10,000 episodes:** the value surface is visibly noisy — some states have only been visited a handful of times, so their return average is unreliable
- **After 500,000 episodes:** the surface is smooth and consistent — as expected, values are high (close to +1) near sum = 20–21 and drop off for lower sums, since the fixed policy keeps hitting until 20
- **Usable ace** states are noisier at 10,000 episodes: they occur less often, so fewer samples accumulate per state for the same number of games

Takeaway: MC estimates are correct in expectation from the start, but their *variance* shrinks only as more episodes are sampled — exactly the bias-free/high-variance trade-off from the theory.

Monte Carlo Estimation of Action Values

Why Q , not V ? With a model, V is enough: look one step ahead over $p(s'|s, a)$ and pick the action with the best reward + next-state value (as in DP). **Without** a model, V alone can't suggest a policy — we must estimate the value of *each action* directly: Q_* , not V_* .

Policy evaluation for action values: estimate $Q^\pi(s, a)$ = expected return from taking action a in state s , then following π . Same MC recipe as for V , but now visits are to a **state-action pair** (s, a) (visited whenever s is visited *and* a is chosen there): first-visit / every-visit MC both converge to $Q^\pi(s, a)$ as the number of visits to (s, a) grows.

The complication: many (s, a) pairs may never be visited. If π is deterministic, only *one* action per state ever gets a return — the others never improve. But comparing actions requires estimates for *all* of them: this is the **maintaining exploration** problem (cf. k -armed bandits).

Exploring starts: a fix — start each episode at a state-action pair chosen so every pair has nonzero probability of being first. Guarantees infinite visits to every (s, a) in the limit, but rarely realistic when learning from real interaction. (Common alternative: stochastic policies with nonzero probability on every action — covered next.)

1. Introduction
2. Monte Carlo Learning
- 3. TD Learning**
4. On-Policy Control
5. Off-Policy Control
6. Summary

From Monte Carlo Averaging to Temporal Difference

Start from the MC incremental update:

$$V(s_t) \leftarrow V(s_t) + \alpha [G_t - V(s_t)]$$

Problem: $G_t = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{T-t-1} r_T$ requires waiting until the *end of the episode* — every future reward.

Key idea: Rewrite the return recursively,

$$G_t = r_{t+1} + \gamma G_{t+1}, \quad \text{and } V^\pi(s_{t+1}) = \mathbb{E}[G_{t+1}]$$

so instead of sampling the true (unknown) continuation G_{t+1} , **bootstrap**: plug in the current estimate $V(s_{t+1})$ in its place.

From Monte Carlo Averaging to Temporal Difference (2)

Substitution: $G_t \longrightarrow r_{t+1} + \gamma V(s_{t+1})$

	Monte Carlo	TD
Target used	G_t (full episode)	$r_{t+1} + \gamma V(s_{t+1})$ (one step)
Needs	episode to finish	only the next transition

→ Same averaging update rule, just a different — and cheaper — target.

Temporal Difference (TD) Prediction

Idea: Bootstrap — use the Bellman equation as a learning rule:

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

Algorithm (TD(0)):

1. For each step: observe (s_t, r_{t+1}, s_{t+1})
2. Compute TD error: $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$
3. Update: $V(s_t) \leftarrow V(s_t) + \alpha \delta_t$

Properties:

- Biased: uses current estimate $V(s_{t+1})$ as target
- Low variance: only one step ahead
- Updates during episodes (online learning)
- More sample efficient than MC

TD(0) — One-Step TD Learning, in Detail

Terminology:

- **TD target:** $r_{t+1} + \gamma V(s_{t+1})$ — what we update $V(s_t)$ towards
- **TD error:** $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ — how wrong the current estimate was
- The “(0)” means **one** step of real reward before bootstrapping — later methods (n-step TD, TD(λ)) look further ahead

Combines the best of both worlds: samples like MC (no model needed) *and* bootstraps like DP (uses an existing estimate) — updates after *every* transition, not just at episode’s end.

Tabular TD(0) for estimating V^π

```
Input: policy  $\pi$  to evaluate; step size  $\alpha \in (0, 1]$ 
Initialize  $V(s)$  arbitrarily, for all  $s \in S$  ( $V(\text{terminal}) = 0$ )
repeat (for each episode)
  Initialize  $S$ 
  repeat (for each step of episode)
     $A \leftarrow$  action given by  $\pi$  for  $S$ 
    Take action  $A$ ; observe reward  $R$  and next state  $S'$ 
     $V(S) \leftarrow V(S) + \alpha [R + \gamma V(S') - V(S)]$ 
     $S \leftarrow S'$ 
  until  $S$  is terminal
until forever
```

[Thinking] Is TD a Controller Steering the TD Error to Zero?

The analogy: $V(s_t) \leftarrow V(s_t) + \alpha \delta_t$, with $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ looks exactly like a proportional controller: δ_t is a tracking error, α is the gain, the update nudges $V(s_t)$ to kill that error.

Where it holds up:

- Step size α = gain trade-off: too large $\rightarrow$ oscillation/instability, too small $\rightarrow$ slow convergence — same intuition as control
- Robbins-Monro conditions ($\sum \alpha_t = \infty, \sum \alpha_t^2 < \infty$) act like a decaying-gain schedule that lets the loop settle
- The “target” $r_{t+1} + \gamma V(s_{t+1})$ itself moves as V updates — closer to an *adaptive/drifting-reference* controller than tracking a fixed setpoint

Where it breaks down: there is no dynamical system being stabilized — $\delta_t \rightarrow 0$ is a *symptom* of V reaching the fixed point of the Bellman operator, not the objective itself. Formally this is a **stochastic approximation / stochastic fixed-point iteration**, not feedback control, even though the update arithmetic matches a P-controller.

MC vs TD Comparison

	Monte Carlo	TD
Update	End of episode	Every step
Bias	None	Yes (bootstraps)
Variance	High	Low
Online	No	Yes
Sample efficiency	Lower	Higher
Convergence	Guaranteed	Guaranteed

Intuition: MC trades high variance for unbiasedness. TD trades bias for lower variance and online learning. Both converge to V^π under right conditions.

Backup Diagrams — Comparing How Far We Look Ahead

Backup diagram: a small tree showing which states/actions an update draws on — the **root** is the state being updated, **deeper nodes** are what its value is computed *from*.

Two independent questions distinguish every method we've seen:

- **Depth** — does the backup bootstrap (stop after one step, plug in an estimate) or go all the way to a terminal state?
- **Width** — does the backup use a *sample* transition, or the *full* expectation over all possible transitions (needs the model)?

→ Next: the three backup diagrams for DP, TD(0), and MC side by side.

Backup Diagram: Dynamic Programming (full backup)

images/backup-dp.png
(missing — add the file)

Depth: one step, but never bootstraps on a *sample* — it uses the true model.

Width: *full* — branches over every action and every possible next state, weighted by $p(s', r|s, a)$.

$$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma V(s')]$$

Requires the model; exact, no sampling variance.

Backup Diagram: TD(0) (shallow sample backup)

images/backup-td.png
(missing — add the file)

Depth: shallow — bootstraps after a *single* real transition, using $V(s_{t+1})$ for everything beyond it.

Width: *sample* — only the one $(s_t, a_t, r_{t+1}, s_{t+1})$ actually observed, no model needed.

$$V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

Low variance, biased, online.

Backup Diagram: Monte Carlo (deep sample backup)

images/backup-mc.png
(missing — add the file)

Depth: deep — the whole episode, all the way to the terminal state; never bootstraps.

Width: *sample* — one single trajectory actually played out, no model needed.

$$V(s_t) \leftarrow V(s_t) + \alpha[G_t - V(s_t)]$$

Unbiased, high variance, needs full episodes.

Backup Diagrams — Side by Side

DP

images/backup-dp.png
(missing — add the file)

full width, shallow

TD(0)

images/backup-td.png
(missing — add the file)

sample width, shallow

MC

images/backup-mc.png
(missing — add the file)

sample width, deep

Punchline: DP and TD both *bootstrap* (shallow); MC and TD both *sample* (no model). TD(0) sits at the intersection — the cheapest possible backup — which is exactly why it scales so well.

1. Introduction
2. Monte Carlo Learning
3. TD Learning
- 4. On-Policy Control**
5. Off-Policy Control
6. Summary

From Prediction to Control

Key insight: Use value estimates to improve the policy.

Policy Improvement: Given V^π , compute:

$$\pi'(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$

Problem: We don't know $p(s' | s, a)$ in model-free setting!

Solution: Work with $Q(s, a)$ instead of $V(s)$:

$Q(s, a)$ = value of action a in state s

Then: $\pi(s) = \arg \max_a Q(s, a)$ — no model needed!

Two approaches:

- **On-Policy:** Learn Q^π while following π (SARSA)
- **Off-Policy:** Learn Q_* while following exploration policy (Q-learning)

On-Policy vs Off-Policy

- **On-policy:** the agent evaluates and improves the same policy π it uses to act.
- **Off-policy:** the agent evaluates or improves a target policy (e.g., π_*) while acting with a different, exploratory policy μ .

On-Policy Monte Carlo Control

Idea: Alternate MC policy evaluation with policy improvement — generalized policy iteration (GPI), but with Q instead of V and sample averages instead of exact expectations.

Monte Carlo with exploring starts (MCES):

1. Start each episode at a random (s_0, a_0) (every pair has nonzero probability) — guarantees exploration
2. Follow current π from there; observe the full episode
3. First-visit MC update: $Q(s, a) \leftarrow$ average of returns following (s, a)
4. Improve greedily: $\pi(s) \leftarrow \arg \max_a Q(s, a)$ for every s visited

Dropping exploring starts — ϵ -soft policies: in real interaction we can't choose the start state. Instead keep π **stochastic**, with $\pi(a|s) \geq \epsilon/|\mathcal{A}(s)|$ for every action (ϵ -greedy is the standard choice): every action keeps nonzero probability, so exploration continues without exploring starts. GPI still holds — improvement stays greedy *within* the ϵ -soft family.

Limitation: converges to the best policy *among ϵ -soft policies*, not the true optimal deterministic π_* .

Monte Carlo Exploring Starts, for estimating $\pi \approx \pi_*$

Initialize $\pi(s) \in \mathcal{A}(s)$, $Q(s, a) \in \mathbb{R}$ arbitrarily, and $\text{Returns}(s, a) \leftarrow$ empty list, for all $s \in S$, $a \in \mathcal{A}(s)$

repeat (for each episode)

 Choose $S_0 \in S$, $A_0 \in \mathcal{A}(S_0)$ randomly so every pair has probability > 0

 Generate an episode from S_0, A_0 following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

for $t = T - 1, T - 2, \dots, 0$ **do**

$G \leftarrow G + R_{t+1}$

if pair S_t, A_t not in $S_0, A_0, \dots, S_{t-1}, A_{t-1}$ **then**

 Append G to $\text{Returns}(S_t, A_t)$; $Q(S_t, A_t) \leftarrow \text{average}(\text{Returns}(S_t, A_t))$; $\pi(S_t) \leftarrow \arg \max_a Q(S_t, a)$

end if

end for

until forever

First-visit MC: the “not in” check skips repeated visits to the same pair within an episode.

On-Policy First-Visit MC Control (ϵ -soft) — Pseudocode

On-policy first-visit MC control (ϵ -soft policies), for estimating $\pi \approx \pi_*$

Algorithm parameter: small $\epsilon > 0$

Initialize $\pi \leftarrow$ an arbitrary ϵ -soft policy; $Q(s, a) \in \mathbb{R}$ arbitrarily; $\text{Returns}(s, a) \leftarrow$ empty list, for all $s \in S, a \in \mathcal{A}(s)$

repeat (for each episode)

 Generate an episode following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

for $t = T - 1, T - 2, \dots, 0$ **do**

$G \leftarrow G + R_{t+1}$

if pair S_t, A_t not in $S_0, A_0, \dots, S_{t-1}, A_{t-1}$ **then**

 Append G to $\text{Returns}(S_t, A_t)$; $Q(S_t, A_t) \leftarrow \text{average}(\text{Returns}(S_t, A_t))$

$A^* \leftarrow \arg \max_a Q(S_t, a)$

 For all $a \in \mathcal{A}(S_t)$: $\pi(a|S_t) \leftarrow 1 - \epsilon + \epsilon/|\mathcal{A}(S_t)|$ if $a = A^*$, else $\epsilon/|\mathcal{A}(S_t)|$

end if

end for

until forever

No exploring starts needed: π always keeps probability $\geq \epsilon/|\mathcal{A}(s)|$ on every action, so exploration is built into the policy itself.

Why ϵ -Greedy Improves Any ϵ -Soft Policy

Claim: for any ϵ -soft policy π , the ϵ -greedy policy π' w.r.t. Q^π satisfies $\pi' \geq \pi$ (policy improvement theorem).

Proof: for any $s \in S$,

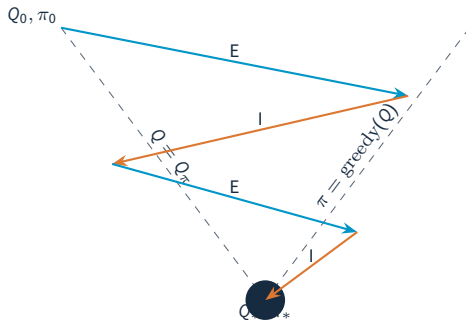
$$Q^\pi(s, \pi'(s)) = \sum_a \pi'(a|s) Q^\pi(s, a) = \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s, a) + (1 - \epsilon) \max_a Q^\pi(s, a)$$

The max is at least as large as any weighted average of the $Q^\pi(s, a)$ with nonnegative weights summing to 1 — in particular the weights $\frac{\pi(a|s) - \epsilon/|\mathcal{A}(s)|}{1 - \epsilon}$, which are ≥ 0 since π is ϵ -soft ($\pi(a|s) \geq \epsilon/|\mathcal{A}(s)|$) and sum to 1:

$$\begin{aligned} &\geq \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s, a) + (1 - \epsilon) \sum_a \frac{\pi(a|s) - \epsilon/|\mathcal{A}(s)|}{1 - \epsilon} Q^\pi(s, a) \\ &= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s, a) - \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s, a) + \sum_a \pi(a|s) Q^\pi(s, a) = V^\pi(s) \end{aligned}$$

So $Q^\pi(s, \pi'(s)) \geq V^\pi(s)$ for every s : by the policy improvement theorem, π' is at least as good as π — restricted to the family of ϵ -soft policies.

Convergence: Monte Carlo Control



Large, infrequent steps: an entire episode is played out (evaluation, E) before the policy is updated even once (improvement, I) — the coarsest possible alternation of generalized policy iteration.

Where it lands: with exploring starts, this reaches Q_*, π_* exactly. With ϵ -soft policies instead (no exploring starts), it converges to the best policy within the ϵ -soft family, exactly like SARSA will next.

SARSA = State-Action-Reward-State-Action (the tuple we observe)

Algorithm:

1. Follow policy π (typically ϵ -greedy)
2. Observe tuple: $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$
3. TD update for Q :

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

4. Improve π : $\pi(s) = \epsilon$ -greedy($Q(s, \cdot)$)

Properties:

- On-policy: learns value of policy being followed
- Converges to Q^π (not necessarily optimal!)
- Conservative: exploration softens the bootstrap target

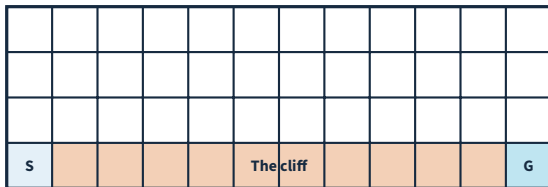
SARSA (on-policy TD control), for estimating $\pi \approx \pi_*$

```
Algorithm parameters: step size  $\alpha \in (0, 1]$ , small  $\epsilon > 0$ 
Initialize  $Q(s, a)$  arbitrarily, for all  $s \in S, a \in \mathcal{A}(s)$  ( $Q(\text{terminal}, \cdot) = 0$ )
repeat (for each episode)
  Initialize  $S$ 
  Choose  $A$  from  $S$  using the policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
  repeat (for each step of episode)
    Take action  $A$ ; observe reward  $R$  and next state  $S'$ 
    Choose  $A'$  from  $S'$  using the policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
     $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$ 
     $S \leftarrow S'; A \leftarrow A'$ 
  until  $S$  is terminal
until forever
```

A' is chosen *before* the update, and is the action really taken next: the bootstrap target uses the real next action, exploration included. That is what makes SARSA on-policy.

Example: Cliff Walking

Setup (Sutton & Barto): a 12×4 gridworld. **Start** S at bottom-left, **goal** G at bottom-right. The rest of the bottom row is the cliff.



Actions: up / down / left / right, deterministic. **Reward:** -1 per step; stepping into the cliff gives -100 and sends the agent back to S (the episode continues).

Same behavior policy for both algorithms: ϵ -greedy, $\epsilon = 0.1$. Same environment, same exploration, only the update rule differs.

SARSA on Cliff Walking — Why It Learns the Safe Path

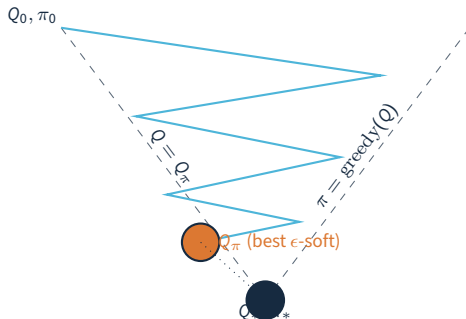
The bootstrap target is the key difference: SARSA's update uses $Q(S_{t+1}, A_{t+1})$, where A_{t+1} is the action *actually* chosen by the ϵ -greedy policy, including the 10% of the time it explores.

Near the cliff: for a state right along the edge, the greedy action is safe, but ϵ -greedy occasionally picks a random action that falls in. SARSA's target feels that risk, because it bootstraps on the action that was really taken, so $Q(\text{edge state, greedy action})$ gets pulled down by the occasional -100 .

Result: SARSA learns to stay away from the edge, a longer but safer path (top row), because it evaluates the value of the policy it actually follows, exploration included.

→ Q-learning, which learns Q_* directly regardless of exploration, behaves differently: compared side by side once it is introduced.

Convergence: SARSA



Small, frequent steps: every single transition alternates one evaluation update ($Q \rightarrow Q_\pi$, along the current policy) with one improvement step ($\pi \rightarrow \epsilon$ -greedy(Q)), much finer-grained than Monte Carlo's per-episode alternation.

Where it lands: because π must stay ϵ -soft to keep exploring, SARSA converges to the best policy within the ϵ -soft family, not the true Q_*, π_* . The gap shrinks as $\epsilon \rightarrow 0$ (with a GLIE schedule, SARSA does reach Q_*, π_*).

1. Introduction
2. Monte Carlo Learning
3. TD Learning
4. On-Policy Control
- 5. Off-Policy Control**
6. Summary

Off-Policy Control — The Idea

Two policies now play different roles:

- **Target policy** π : the policy we want to evaluate or improve, usually the greedy / optimal one
- **Behavior policy** μ : the policy that actually generates the data, usually exploratory (e.g., ϵ -greedy)

Example: you learn to play chess by *watching* an amateur play thousands of games, sometimes blundering, sometimes brilliant. You never move a piece yourself, yet from those games you want to figure out the value of the *best possible* play, not the value of the amateur's play.

- Behavior policy μ = the amateur's exploratory, imperfect play
- Target policy π = the optimal play you are trying to learn about

Why bother? Reuse old or third-party data, learn about several target policies from one stream of experience, or keep exploring while still learning the optimal answer.

Importance Sampling — Correcting for a Different Policy

Requirement (coverage): every action the target policy might take must also be possible under the behavior policy:

$$\pi(a|s) > 0 \implies \mu(a|s) > 0 \quad \text{for all } s, a$$

General tool: reweight experience collected under μ to estimate an expectation under π , using the **importance-sampling ratio** of a trajectory from t to $T - 1$:

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{\mu(A_k|S_k)}$$

Why it works: reweighting the return G_t actually observed under μ by $\rho_{t:T-1}$ gives an unbiased estimate of the value under π :

$$\mathbb{E}_{\mu}[\rho_{t:T-1} G_t \mid S_t = s] = V^{\pi}(s)$$

Importance Sampling — Ordinary vs Weighted

Ordinary importance sampling: plain average of the reweighted returns over N visits to s :

$$V(s) = \frac{1}{N} \sum_{i=1}^N \rho_{t_i:T(i)-1} G_{t_i}$$

Unbiased, but ρ is a product of ratios: it can blow up or vanish over a long episode, so the variance can be very large, even unbounded.

Weighted importance sampling: normalize by the sum of the ratios instead of N :

$$V(s) = \frac{\sum_{i=1}^N \rho_{t_i:T(i)-1} G_{t_i}}{\sum_{i=1}^N \rho_{t_i:T(i)-1}}$$

Biased for finite N (asymptotically unbiased), but its variance stays bounded, so it is what is used in practice.

Takeaway: explicit importance sampling is the general recipe for off-policy prediction, but its variance is a real cost.

From Importance Sampling to Q-Learning

Q-learning's shortcut: pick the target policy to be exactly greedy w.r.t. Q :

$$\pi(s) = \arg \max_a Q(s, a).$$

For a greedy target, the expectation under π of the next action's value collapses to a single term:

$$\mathbb{E}_{A' \sim \pi} [Q(s', A')] = \max_{a'} Q(s', a')$$

Consequence: the bootstrap target $\max_{a'} Q(s_{t+1}, a')$ *already is* the value under the target policy, computed directly instead of estimated by reweighting samples from μ . No ratio ρ , no extra variance, and μ can be anything that keeps visiting every (s, a) often enough.

→ That is why Q-learning, not general off-policy MC/TD with importance sampling, is the standard off-policy control algorithm in practice.

Q-Learning = Learn optimal Q_* while exploring

Algorithm:

1. Follow any exploration policy μ (e.g., ϵ -greedy)
2. Observe tuple: $(s_t, a_t, r_{t+1}, s_{t+1})$
3. TD update — always take max in target:

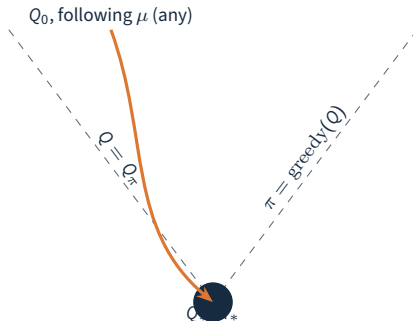
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

4. π is greedy w.r.t. current Q ; we explore with ϵ -greedy

Properties:

- Off-policy: learns Q_* despite exploring
- Converges to Q_* (optimal action-value)
- More aggressive: target is always the best action (max)
- Can reuse off-policy data

Convergence: Q-Learning



One update, two roles at once: the $\max_{a'}$ in the target fuses evaluation and improvement into a single step, so Q is pulled directly toward Q_* regardless of which policy μ generates the data.

Why the trajectory can leave the cone: unlike SARSA, the update never needs μ to equal, or even resemble, the greedy target policy; μ only has to keep visiting every (s, a) often enough. That is exactly why Q-learning reaches the true optimum while exploring freely, and can reuse data from any exploratory behavior.

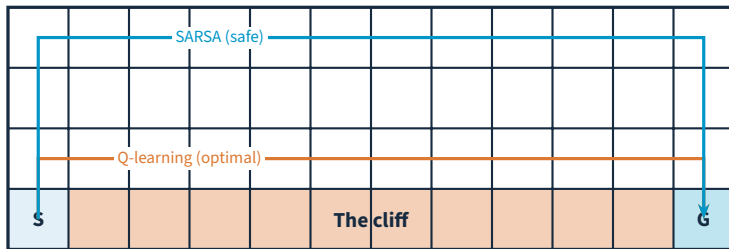
SARSA vs Q-Learning

	SARSA	Q-Learning
Policy	On-policy	Off-policy
Bootstrap target	$Q(s', a')$ (actual)	$\max_a Q(s', a)$ (best)
Converges to	Q^π	Q_*
Behavior	Conservative	Aggressive
Data reuse	No	Yes

Rule of thumb:

- **SARSA:** When you want to learn safely (respects the actual policy)
- **Q-Learning:** When you want the optimal policy (doesn't matter how you explore)

Back to Cliff Walking: SARSA vs Q-Learning



Q-Learning (hugs the cliff): learns Q_* directly, so the greedy path it converges to is the shortest one, right along the edge, even though the ϵ -greedy *behavior* still occasionally falls in during training.

SARSA (top row): learns the value of the policy it actually follows, exploration included, so it prefers the longer path that keeps a safety margin.

Sutton & Barto (2018), Example 6.6 — online, Q-learning actually collects a *lower* total reward during training than SARSA, precisely because it keeps risking the — 100 cliff step while learning.

1. Introduction
2. Monte Carlo Learning
3. TD Learning
4. On-Policy Control
5. Off-Policy Control
- 6. Summary**

Convergence: MC vs TD vs DP

	DP	TD	MC
Requires model	Yes	No	No
Online	No	Yes	No
Sample efficient	N/A	High	Low
Bias	No	Yes	No
Variance	N/A	Low	High
Convergence	Guaranteed	Guaranteed	Guaranteed

Summary:

- **DP:** Perfect model, fast exact solution (Lecture 2)
- **TD:** No model, online, sample-efficient, biased (sweet spot)
- **MC:** No model, unbiased, high variance, episodic

Challenge: How to explore (find better policies) while exploiting (use best known action)?

ϵ -Greedy Strategy:

- With probability ϵ : take random action (explore)
- With probability $1 - \epsilon$: take best known action (exploit)
- Simple, effective, easy to implement
- Typically $\epsilon \approx 0.1$ (10% exploration)

Other exploration methods:

- Boltzmann: soft-max over values
- UCB: optimism under uncertainty
- Thompson sampling: posterior sampling

Key point: Without exploration, never find better policies. But too much exploration wastes time.

Exploration Across Control Methods

	MC Control	SARSA	Q-Learning
Exploration source	Exploring starts, or ϵ -soft π	ϵ -soft π	Any μ with full coverage
Needed for	Correctness (data coverage) <i>and</i> safety	Correctness (data coverage) <i>and</i> safety	Only coverage of every (s, a)
Converges to	Q_*, π_* (ES) / best ϵ -soft policy	Best ϵ -soft policy	Q_*, π_* (exact)
Needs $\epsilon \rightarrow 0$?	Yes, unless ES	Yes (GLIE)	No
Cliff Walking	—	Safe path (top)	Optimal path (edge)

The pattern: on-policy methods (MC control, SARSA) bake exploration *into* the policy they evaluate, so they end up conservative near danger. Q-learning decouples the two: it can explore as aggressively as needed for coverage without that risk ever entering the value it converges to.

Key Takeaways

- **Prediction:** MC (unbiased, high var) vs TD (biased, low var, online)
- **Control:** SARSA (on-policy, conservative) vs Q-Learning (off-policy, optimal)
- **Trade-off spectrum:** DP $\rightarrow$ TD $\rightarrow$ MC in terms of bias/variance/sample-efficiency
- **Exploration:** Essential for learning; ϵ -greedy is simple baseline
- **Convergence:** All guaranteed under appropriate step sizes and exploration
- **Practical:** TD + ϵ -greedy often best balance for real problems

Questions?

Your feedback on this lecture is welcome

etienne.chassaing@centralesupelec.fr



Survey form (anonymous feedback):

<https://forms.gle/L7ewxBUTUaZbpcya8>



Next: Lecture 4 — Advanced topics and applications!

About this course: I do use LLMs/AI to create this course, with the following uses:

- Format $\text{\LaTeX}$ slides and exercises
- Find relevant sources
- Format equations and notations

However, plan and ideas are human-based, and every content has been carefully reviewed. If you find some errors, or inconsistencies, please report them to me.

What about your own LLM usage?

Of course you also have the right to use LLMs, but don't forget you're in a learning step. Learning process goes through effort and the newer the subject, the more one needs empty-sheet recall.

* See: Desirable difficulty

* Chi & Wylie (2014), ICAP hypothesis